SIMD/Vectorization

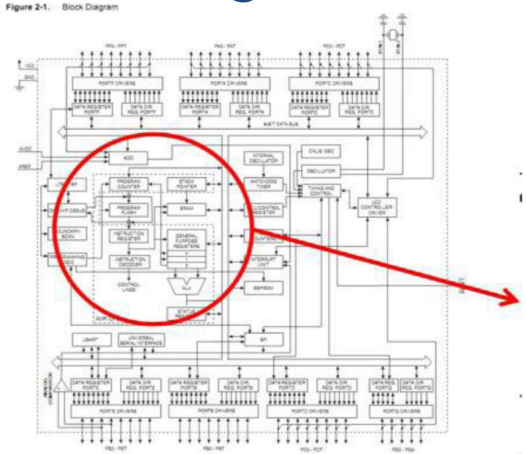
By: Sean Chang

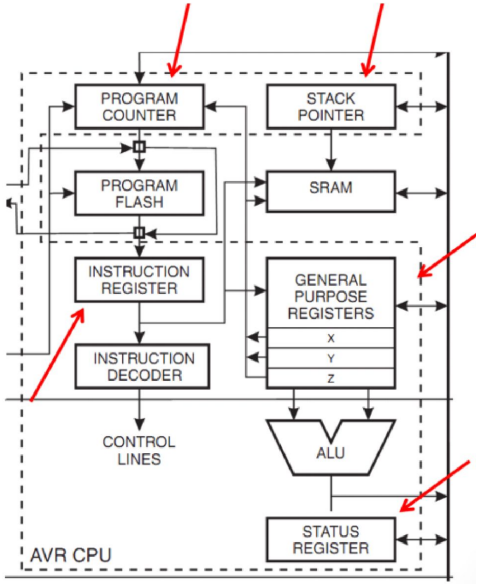
What Are Registers?

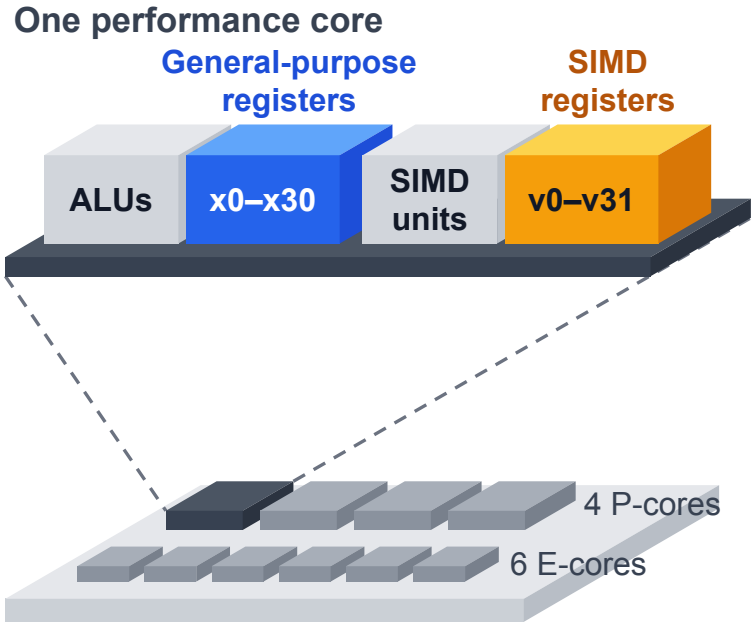
- Small, ultrafast blocks of memory right next to logic units
- Where intermediate results/flags are usually stored (for ALU, for example)
- **Regular(general purpose) registers** correspond to the x-bit of your machine (32-bit, 64-bit on mine)
  - Denoted in assembly by “x0-x30”
- Also have **simd/floating registers,** which are extra-wide registers for SIMD operations (128-bit on my Apple, up to 512-bit on x86)
  - Denoted by “v0-v31”
- On x86:
  - Xmm - 128
  - Ymm - 256
  - Zmm - 512

Where Are They?

CPU Registers







Apple M4 chip (schematic, not to scale)

What Does Using them Look Like In Code?

Assembly:

- b0 is the low 8 bits of v0
- h0 is the low 16 bits
- s0 is the low 32 bits
- d0 is the low 64 bits
- q0 is all 128 bits

Lanes:

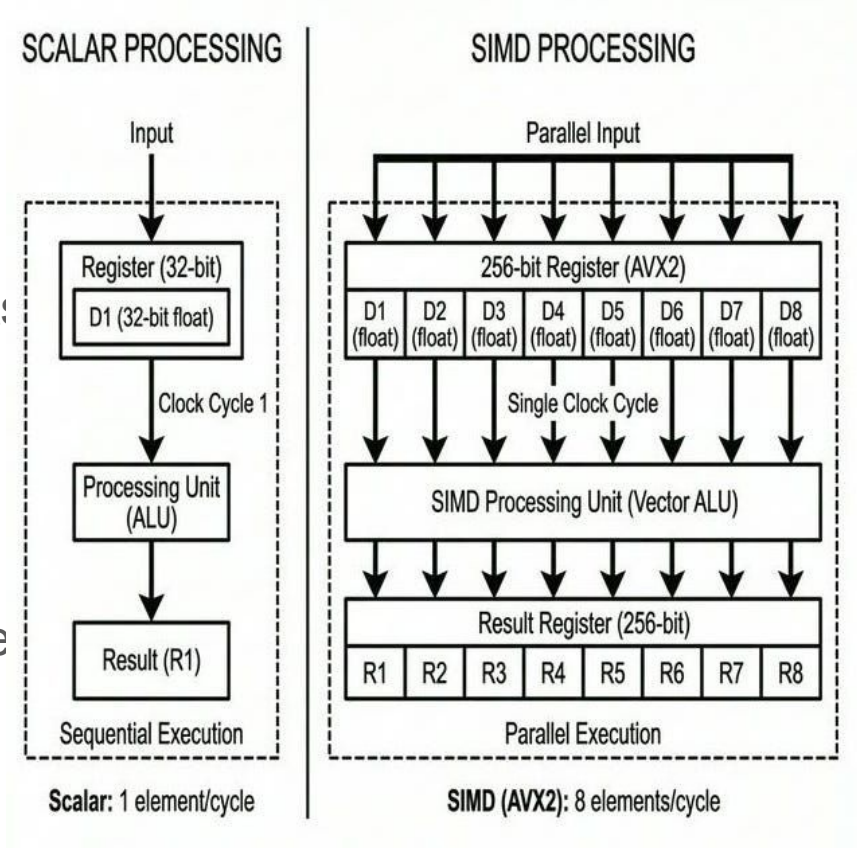
- Split registers into “lanes” to perform operations on individually (so you don’t waste 128 bits on 32 bits of data)

Ex:

- `ldr s0, [x8, x9, lsl #2] vs. ldp q0, q1, [x12, #-32]`

What is SIMD Processing?

- Performing operations on the whole simd register
- Allows for processing of multiple element one cycle
- Ex: Instead of using 4 instructions to add 32-bit floats, you use 1 to add 4 of them
- Hence “single instruction, multiple data”
- Note: Helps in **both** the *load* step and the *process* step.
- Limitations: if memory is scattered, loads be vectorized



Practical Applications

- Database querying (walking down a column)
- AI (matrix multiplication)
- Video rendering (also matrix multiplications)
- Strlen (check all chars)
- Compression (find repetitions/patterns)
- And much more!

How Much Does it Help?



```c
#define SIZE 1000000
#define ITERS 10000

static inline void keep(void *p) { __asm__ volatile("" : : "r"(p) : "memory"); }

int main() {
    int32_t* v1 = malloc(SIZE*sizeof(int32_t));
    int32_t* v2 = malloc(SIZE * sizeof(int32_t));
    int32_t* v3 = malloc(SIZE*sizeof(int32_t));
    for (int i = 0; i<SIZE; ++i) {
            v1[i] = rand();
            v2[i] = rand();
    }
    int64_t sum = 0;
    clock_t start = clock();
    for (int j = 0;j<ITERS;++j) {
        for (int i = 0; i<SIZE; ++i) {
            v3[i] = v1[i] + v2[i];
        }
        keep(v1);
        keep(v2);
        keep(v3);
    }
    clock_t end = clock();
    free(v1);
    free(v2);
    free(v3);
    double elapsed = (double)(end-start) /CLOCKS_PER_SEC;
    printf("Average Time: %.10f microseconds\n", (elapsed*1000000) / ITERS);
    return 0;
}
```

Example 1 (a program that fits SIMD well). Average times:

- O0: 903.17 microseconds
- O1: 260.51 microseconds
- O2: 100.00 microseconds
- O3: 99.99 microseconds



```c
#define SIZE 1000000
#define ITERS 10000

static inline void keep(void *p) { __asm__ volatile("" : : "r"(p) : "memory"); }

int main() {
    int32_t* idx = malloc(SIZE*sizeof(int32_t));
    for (int i = 0; i<SIZE; ++i) {
        idx[i] = i;
    }
    for (int i = SIZE-1; i>=0; --i) {
        int k = rand() % (i+1);
        int32_t t = idx[i];
        idx[i] = idx[k];
        idx[k] = t;
    }

    int32_t* v1 = malloc(SIZE*sizeof(int32_t));
    int32_t* v2 = malloc(SIZE * sizeof(int32_t));
    int32_t* v3 = malloc(SIZE*sizeof(int32_t));
    for (int i = 0; i<SIZE; ++i) {
            v1[i] = rand();
            v2[i] = rand();
    }
    int64_t sum = 0;
    clock_t start = clock();
    for (int j = 0;j<ITERS;++j) {
        for (int i = 0; i<SIZE; ++i) {
            int k = idx[i];
            v3[k] = v1[k] + v2[k];
        }
        keep(v1);
        keep(v2);
        keep(v3);
    }
    clock_t end = clock();
    free(v1);
    free(v2);
    free(v3);
    double elapsed = (double)(end-start) /CLOCKS_PER_SEC;
    printf("Average Time: %.10f microseconds\n", (elapsed*1000000) / ITERS);
    return 0;
}
```

Example 2 (a program that does not fit SIMD well) Average times:

- O0: 2347.43 microseconds
- O1: 1753.33 microseconds
- O2: 1777.84 microseconds
- O3: 1813.72 microseconds

Interesting Discovery

```
static inline void keep(void *p) { __asm__ volatile("" : : "r"(p) : "memory"); }
```

- You may have noticed this function
- It tells the compiler that something may read the arrays
- -O2 deletes vectors entirely if they’re not used
- “As-if” rule - just needs to preserve observable behavior

```
for (int i = 0; i<SIZE; ++i) {
        v1[i] = rand();
        v2[i] = rand();
}
```

- Also had to randomize values otherwise compiler cached a single value and just rescaled it inside the register every time

Why?

Example 1 Asm:

O0

```
ldur    x9, [x29, #-24]
ldrsw   x10, [sp, #32]
ldr w9, [x9, x10, lsl #2]
add w8, w8, w9
```

O2

```
ldp q4, q5, [x11]
ldp q6, q7, [x11, #32]
add.4s  v0, v4, v0
add.4s  v1, v5, v1
add.4s  v2, v6, v2
add.4s  v3, v7, v3
```

Example 2 Asm:

O0

```
ldur    x9, [x29, #-48]
ldrsw   x10, [sp, #52]
ldr     w9, [x9, x10, lsl #2]
add     w8, w8, w9
```

O2

```
ldr     w5, [x21, x15, lsl #2]
ldr     w6, [x21, x16, lsl #2]
add     w17, w3, w17
add     w0, w4, w0
add     w1, w5, w1
add     w2, w6, w2
```

In Conclusion

- Write programs that are optimized for vectorization by making accesses and operations predictable
- Make sure to compile program with -02 or -03 for >10x speedup in good cases
- Be aware of *what* kind of operation your program does most, and focus your optimization efforts on the biggest workflows.